

The Global Examiner

VOL. 1, NO. 078

2026-03-30

FEATURES

How to get a job at FINN.no

espen · Finn.no Tech

If you're a student or just someone who is looking for a job you might be wondering: "How could I get to work for FINN.no?". One guy found an answer and is now one of the key developers on our iOS applications. We did an advert for summer interns last year and he was one of the people who came in to do some proof of concept work on iOS. The end result was an iPad application that never hit the AppStore, but for one of the participant this internship was the first step towards landing a job as a developer at FINN.no.

What made you apply for the summer internship at FINN.no?

It all started with my university teacher pointed out that FINN.no had an open internship during the summer. At first I did not want to apply, because I wanted to have vacation all summer (Like all other informatics students). However, I ended up with applying and just a few hours later I got a call; the internship was mine! The main reason to apply for the internship was that I was finishing my master's degree and was lacking work experience in the field of informatics. Secondly, I did not know that much about FINN.no and I could always need the extra money :)

What did you learn during that summer?

I learned that FINN.no is not just a big website, it is also a big company. And there are not just developers at FINN.no, but also a range of different jobs (Like finance, marketing, economics and so on). When taking about the technical aspects I learned how to develop for the iPhone/iPad. Furthermore, I learned that to get things working you are depended on a range of different "teams" to develop what you need. And this is not always trivial, because all "teams" have their own focus areas and problems. So you never work independent!

What was it like to be a summer intern at FINN.no?

It was both interesting and rich on experiences. The one thing I want to point out is how the atmosphere and the peoples at FINN.no are. The atmosphere is comfortable and "chill" and the people are welcoming and easy to talk with. They want to teach you what they know and want to learn about what you know.

How did you do about landing the position at FINN.no?

Firstly, I work hard during the summer and tried my best to present what I could do. Before I left FINN.no that summer I was told that if I needed a part time work, I should just ask and they would see what they could give me. After the summer internship was completed I went back to school for a few months. Some time before Christmas I sent my supervisor at FINN.no a e-mail and ask if it was possible to get a 50% position at the company the last 6 months of my masters degree. 1. January I started with working part time at FINN.no and before I completed my degree I was presented with a full time position at FINN.no.

What have you been doing since you came to work for FINN.no?

I have been continuing learning objective-c and iOS. Furthermore have I learned more about designing projects, completing them and how to work with other development teams (To get our hands on the right APIs we needed in the iOS app). One example is the new iOS app, FINN Torget annonse innlegging, that FINN.no released a week ago.

What is the difference between working with school assignments and working at a company?

The biggest difference must be that a school assignment is something that will never be used by any persons. So you just complete it because you must, and not because you want to. In a company as big as FINN.no your applications will be used by a lot of peoples. You must always perform your best to please all of the users. Because of this you want to stay on the edge of new technology and learn more.

What's your advice for other young developers out there wanting to work for the coolest company in Norway?

You must work hard and make them see you in the crowd. If a new internship opens up do not think twice just apply. Before you know it, you are placed in my position to answer a few questions in a interview like this one.

Anything you want to add?

I love how they work at FINN.no, a lot of very smart people and it feels like peoples at FINN.no don't stress that much! And peoples at FINN.no do know how to create a party ;)

Beyond the Mean

GameChanger · GameChanger Tech

I work as a software engineer at a sports tech company and I am a woman. These facts about my life are why I continue to be distressed about the news of the Google Memo, written by software engineer James Damore, and its ramifications for my industry. While the headlines may make the memo seem like it was written by a deranged maniac, the fact that it wasn't written as a blatant anti-diversity statement (it begins: "I value diversity and inclusion, am not denying that sexism exists, and don't endorse using stereotypes") is one of the things that makes it harder for people to be immediately outraged about its contents (and why this discussion is so fascinating and important).

As a software engineer, questioning assumptions and raising potentially controversial ideas is part of the job. I agree with the Damore's assertion that "If we can't have an honest discussion about this, then we can never truly solve the problem." The topic of diversity is critically important and I want to add my voice and insights to the discussion as honestly as possible. I agree with the myriad of responses suggesting that there were many productive ways that Damore could have raised some of his concerns, such as his views that conservative viewpoints aren't valued in tech or that the diversity programs at Google should be more inclusive, without perpetuating gender stereotypes to make his point. I am pained to see that people are now afraid to raise controversial or minority opinions or ask questions because of the consequences of this memo.

But the issue for me is not the theoretical or hypothetical raising of unpopular opinions. In theory I absolutely agree that is very important. But I want to focus on the reality of this situation and the memo that was actually published. Damore's main argument is that "Google's political bias has equated the freedom from offense with psychological safety, but shaming into silence is the antithesis of psychological safety." However, you can't ask for psychological safety for one group while stomping on the psychological safety of another group. Content matters. The way these opinions were raised matters. Making an entire gender feel less qualified or question their career choice matters. As a female software engineer, the response to this memo makes me feel less safe. As a coworker, and fellow female engineer, put it, "[the memo] weighs on me because it validates my fears that I will have to spend most of my career proving I am competent before I'm given the space and respect to learn and grow."

Despite Damore specifically recommending against reducing a population to their average, by even discussing "average" traits of women and men, the memo now provides specific stereotypes for people to point to to explain why a woman may be a lesser engineer. When you first start at a job or on a team no one has reasons to assume that your traits are anything but "average" until proven otherwise. If people believe that the "average" woman is less qualified it puts more onus on women to prove their worth the second

they walk in the door. And trust me, even without this memo circulating, women in the tech industry are constantly being forced to prove their competence.

Damore selected three particular traits to discuss. He did not include his reasons for selecting those particular traits as his examples of genetic differences between males and females, although he did discuss ways in which, in his opinion, those traits cause problems in the tech industry. In summary, he mentioned that "average" female traits include: 1) "openness directed towards feelings and aesthetics, rather than ideas...[including] a stronger interest in people rather than things;" 2) "extraversion expressed as gregariousness rather than assertiveness," which included "higher agreeableness;" and 3) "neuroticism (higher anxiety, lower stress tolerance)." Each of these terms in the memo also hyperlinks to a Wikipedia definition, rather than scientific treatises, and therefore seem to be a random selection of traits and stereotypes that Damore believed to be detrimental to the workplace. However, not only am I skeptical of the representativeness of the chosen traits and the sources cited, I believe that some of these traits are actually crucial to create a balanced, productive workplace. Imagine software built by a team where all members were more interested in "things" than people and no one wanted to pay attention to aesthetics (assuming the team was able to ship anything if it was comprised of assertive rather than agreeable people).

Additionally, while defining these "average" traits, Damore's stated that "More men may like coding because it requires systemizing and even within [software engineers], comparatively more women work on front end, which deals with both people and aesthetics." This assertion really got under my skin because, for a split second, it made me self-conscious of being a front end leaning developer. I do not only enjoy front end work because of aesthetics, that is one of the last pieces that I consider in my day to day work. Working as an engineer in any capacity requires deep systemizing and writing code. Contrasting front end work with coding in this way undermines the amount of critical thinking that front end engineering requires. This also serves to enforce biases that one gender should work on a particular part of the stack.

I can only share my own experience, but I believe it is important for more individuals to share their stories. To show that generalizing to the "average" actually hurts real people on real journeys.

My entire adult life I have heard things like "you only got into MIT because you're a woman" or "you're just a diversity hire." I went to a job interview where I asked how many women they had on the engineering team and the interviewer freaked out briefly and said "We only have one! But we're trying to improve that number! That's why we're interviewing you!" By citing a diversity statistic as a reason to interview me, I suddenly felt less qualified for the position. Needless to say, I did not accept that job offer. But this constant questioning or suggesting that I am only interesting because I increase diversity numbers leads to

When flying from Oslo to London, you don't want to go via Moscow

Per Jørgen Walstrøm · Finn.no Tech

If you want to travel from Oslo to London, you may end up getting results which include layovers in far-away places. Do you really want to spend a night at the airport in Moscow if you only want to go from Oslo to London? Probably not, but nonetheless, some of the results we receive at the FINN Flight Search are those kind of travels. Those travels may be really cheap, ending up at the top of our result list as we sort by price, but they are mostly garbage and we do not want our users to be confused by those outliers. We want them to go away!

The old solution

We have tried to fix this before. With limited success. A solution we used a couple of years back was something like this:

$x = 3 * \text{Average of the three shortest flights}$

We removed results with duration $> x$. This works fine on short travels, like a two-hour flight from Oslo to London. With the given formula, all trips longer than 6 hours to London will be removed. That's ok.

However, it doesn't work too well on longer flights. If you want to go from Oslo to Bangkok, you would probably spend at least 11 hours in the air. In that case, it would be too conservative to just remove the flights that are 33 hours or more.

A theoretical solution

The correct way to remove the outliers is by finding the quartiles. When the quartiles are found, the upper fence is defined by

Upper fence = $Q1 + 1.5 * IQR$

Where IQR is the Interquartile Range; $Q3 - Q1$.

We are using Solr at FINN, but the stats query in Solr does not give us the quartiles out of the box, so we would need to do a separate query to calculate the quartiles. Should be easy enough.

What is too long?

So what is a too long flight? That's not an easy question to answer. Racking our brains (and doing some guesswork), we came up with the following suggestions on what the upper fence on duration to different destinations should be. The durations are per leg:

OSL - LON 300 mins = 5 hrs. Given the minimum flight time
OSL - LON = 115 OSL - BKK (Bangkok) 1000 mins = 17 hrs.
Minimum = 674 OSL - KOA (Kona, Hawaii) 2000 mins = 33 hrs.
Minimum = 1335

The minimum flight duration is clearly of interest to us. We store that away in Solr, and using the stats query from Solr on that field, we get the following data on a flight OSL-LON:

```
"stats_fields": { "minimumLegDuration": { "min": 115, "max": 815, "count": 3610, "missing": 36, "sum": 615655, "sumOfSquares": 123968775, "mean": 170.5415512465374, "stddev": 72.5080446083144, "facets": {}
```

A lot of good information there! We get the standard deviation, mean and min values. Missing the quartiles, though, but we can fix that!

And what is too short?

Sometimes, the supplier sends us incorrect data. That's bad. But the good thing is that when the data is incorrect it is really incorrect. We sometimes receive flight times of 0 minutes. That is obviously wrong, and we filter out those. However, we have not seen any occurrences of, say, 1 minute flight times. That would be wrong as well, but more difficult to handle. So we close our eyes and don't care about those.

But hang on...

So, we were ready to go! We wanted to fix this problem once and for all. But before we started coding, we talked to emeritus Harald Goldstein at the University of Oslo. He's been teaching statistics at the university level for years and had some good feedback on our theoretical solution. First of all, he pointed out that the solution we have outlined is a good solution if we have a symmetrical distribution of the flight durations. The quartiles and the standard deviation works well on a symmetrical distribution like the normal distribution, but what about our data? Is it normal distributed? That was a relevant question, and as one of FINN's main strategies is "data in our backbones", we surely needed to have a closer look at this.

So here goes, flights from Oslo to London. Trip duration on the x-axis, number of occurrences on the y-axis:

The shortest trip duration OSL - LON is 230 minutes in the graph above (115 min per leg) and there is even an outlier - which is not visible - at 2750 minutes! However, it is quite clear that this is not a normal distribution. It might look more like a gamma distribution for the part where $x > 230$.

Darn!

Back to square one

All the fancy statistics didn't provide the solution, but it helped us really understand the problem. Obviously, we needed to re-think.

A problem with the solution from ages back (described at the beginning), was that it didn't work very well on long hauls. The relation between the minimum trip duration OSL-BKK and OSL-LON is approx. 6:1. Alas, the upper fence must be relatively lower on longer trips. After some more brain racking, we came up with the idea that we should look

Hello world!

Per Jørgen Walstrøm · Finn.no Tech

As some of you may know, FINN is owned by Schibsted. What may not be that familiar, is that Schibsted has online classified solutions in several markets around the world, including Blocket in Sweden, Le Bon Coin in France and Vibbo in Spain. Schibsted has several hundreds of developers distributed around the globe, and there is a strong need and will to coordinate and cooperate across geographical boundaries.

The need to cooperate

Reinventing the wheel is generally a bad idea. With that in mind, Schibsted has a strong focus on building global components that can be useful for all the companies within Schibsted. However, it is not always easy to understand the needs and challenges across national borders. There are cultural differences and different companies have reached different maturity levels. FINN has been around for years, and is one of the more mature classified sites within Schibsted. In an effort to help building global components, Schibsted has set up an international hub in Barcelona and developers from the local companies are encouraged to come to Barcelona to help out, share knowledge and build a common platform. Several developers from FINN have taken up the challenge and spent some months in beautiful and vibrant Barcelona. Here are Øyvind's impressions.

Anders, Øyvind and Per Jørgen. Three happy FINN-campers in Barcelona

Øyvind goes messaging

Øyvind has worked as a developer for FINN for 9 years, but when he got the opportunity to move to Barcelona for 6 months, the decision was easy.

- The decision was really easy. There was little risk involved and I thought it would be an excellent opportunity to experience Barcelona. I love the city and the international atmosphere at the office. I think we are around 25 nationalities working here.
- Tell us about the work you have been doing in Barcelona
- I work as a developer here, in a team that develops a global messaging-component to be used by Schibsted-owned classified-sites around the world. When I arrived in January, we had integrated 6 global sites and the platform handled hundreds of thousands messages/day. Today, we have even more sites using our platform. It's worth mentioning that the messaging-platform I'm working on, originally was developed by FINN. In 2014, however, it was decided that this platform was to be promoted to become a global component, so the responsibility was moved from Oslo to the global components team in Barcelona. The developers in Barcelona inherited the FINN code-base and developed it from being a FINN-only solution to a full-fledged multitenant solution. It's

not easy to inherit code like that, but I feel the ownership has been transferred now. The Barcelona-team owns the product and is soon going to integrate with FINN as well. We'll replace FINN's messaging platform with the global component-version, but that is not an easy task. FINN needs to preserve it's messaging history, and that means we have to import 150 million messages! It's a big job for our Cassandra-nodes.

- What are the technical differences compared to working in FINN?
- We use Amazon Web Services and it is really great to see how easy it is to scale up and set things into production. The infrastructure team here in Barcelona has done an amazing job and I feel it is really at the bleeding edge of technology working in this environment.
- You are soon returning to Norway. What will you miss from your stay in Barcelona?
- The people! It has been great working here, in this including atmosphere.
- Learned any lessons?
- When working in FINN, it's easy to forget that you are part of a bigger company. I see that clearly now, the global perspective. It's inspiring to know that I am a part of a global company.

Øyvind spent 6 months in Barcelona and came home with lots of new knowledge, new friends and a Messi-shirt. He contributed in building global components and believes FC Barcelona will win Champions League again this season. The migration of the FINN-messaging platform went off without a hitch and we are still looking for great developers!

React in the Real World

Tor Arne Kvaløy · Finn.no Tech

At FINN oppdrag , a year ago we came to realize that our front-end code, written in JSP/jQuery, had become so large and complex that it affected our ability to produce features. We spent more time making sure the DOM manipulations were correct and did not break, than actually creating value for our users.

So we agreed that we needed a front-end framework to simplify our development and after looking into Ember and Angular we decided to go for React , recently open sourced by Facebook.

Our attraction to React was based on its promise of providing components with encapsulated state, fast automatic DOM manipulations, and the ability to render on the server.

We started by porting some functionality that already existed, and our first discovery was that the lines of codes needed to make the exact same functionality was almost reduced by half. Secondly, the code became clearer and easier to understand. And thirdly, as a combination of the reduced lines of code, the cleaner code and no manual DOM manipulations, we got less bugs.

Since then all new features have been written with React, and when we are requested to make modifications to functionality written in the old code, we rewrite them to use React.

When migrating large pages to React we start from the inside, the deepest DOM nodes, and let it gradually grow till the whole page is reactified. This often happened through several iterations and deployments, as React and the old JSP/JQuery can quite peacefully coexist.

With React, HTML is represented as JavaScripts, so the JavaScript bundles will contain all the HTML, and hence grow as the pages grow. Instead of having a bundle for each page (very small), or having a bundle for the whole site (very big), we found a middle way where we have bundles based on roles, in our use cases contractor and owner. A possible improvement on this is loading JavaScript on demand with technologies like webpack .

So if React is our first great discovery, the Flux pattern is very much in second place. The Flux pattern decouples the logic from the views, in a different way than the traditional MVC-pattern. Logic and state are located in stores, and actions from the views to stores are asynchronous. When state in stores change, they will update React components that listen to changes, providing a single source of truth and unidirectional flow of data.

Server-side rendering

Our next step, which is yet to be solved, is server-side rendering. Server-side rendering means having the server generate and return the whole HTML page to the browser,

instead of having the browser render HTML after downloading the JavaScript. This is especially important for SEO (search engine optimization) and mobile users, as search engines rarely run JavaScript, and mobile users have low bandwidth hence will take longer time before the page shows. However, having the server return a fully rendered HTML to browser is in general good for perceived page load performance.

Server-side rendering requires running JavaScript on the server, and that is straightforward with Node. However it is not that straightforward in our Java/Tomcat/Spring backend stack. We envision three possible solutions for this problem: 1) Running JavaScript on the JVM with the Nashorn JavaScript engine. 2) Delegating execution of JavaScript to an external process running Node. 3) Having Node run as smart-proxy in front of Java/Tomcat.

All these three solutions have their own benefits and challenges.

The first solution, running JavaScript in Java, has the benefits of close integration with the controller layer written in Java, and possibly making deployment easier, as the Java environment already has access to the JavaScript bundles. The downside of this approach seems to be performance, as the Nashorn JavaScript engine compiles and optimizes the JavaScript during the first runs, hence uses a long time to «warm up». So unless we write a mechanism that warms up the code, the first pages will be very slow to load.

The second solution, which involves interacting with an external process running Node, has the benefits that JavaScript runs natively and very fast. The downsides are more complicated deployment process where the Node instance needs access to the JavaScript bundles, and the overhead of interacting with an external process, possible through HTTP or Thrift, can impact performance negatively.

The third solution, where Node is placed in front Tomcat, and works as a smart-proxy , only enhancing certain pages, while simply forwarding the rest, has the benefits of executing JavaScript natively. While the downsides are more complicated deployment, and the possible complex process of executing certain JavaScript code found in HTML, and updating the HTML with the executed JavaScript, before it is served to the browser.

So, apart from server-side rendering, which we are working on, our transition to React has been smooth and a great success. It has improved our codebase, made developers happy, and most importantly, improved our speed of delivering features, which in the end is the most important thing.

Tor Arne Kvaløy Senior developer

Graded browser support at FINN.no

espen · Finn.no Tech

Inspired by the kick-ass developers of Yahoo! and their work on Graded Browser Support (GBS) we at FINN.no decided to adopt graded browser support as a way to communicate what level of support we have for different combinations of browsers and platforms.

Creating the support matrix

FINN is the largest site in Norway when it comes to traffic and we have a good framework for statistics. In order to create the support matrix we took the statistics for the most popular browsers and platform combinations and put them in a grid. This gave us a matrix showing us which browser/platform combinations we need to consider from a business perspective. Although this was a good start, we also needed to figure out what level of support we should provide each combination. Numbers and usage alone does not provide a good enough basis upon setting support levels.

The cost of support

Having read the early drafts of the awesome book *Secrets of a JavaScript ninja* by John Resig we decided to follow his approach on creating a GBS matrix and perform a cost-benefit-analysis. We did was to do a quick survey among our developers in order to figure out what the costs where for supporting different user agents on certain platforms. The results are displayed in the chart below and shows our sub-

jective opinions on how much effort is required to support a certain browser. Note that this chart will not look the same for everyone. It is will vary based on the skills of your developers, what browsers they work in, etc, etc.

Support levels

Our levels of support are slightly more simple than those of Yahoo! and we only provide three levels:

A-support: no visual or functional errors, all errors are reported and all features should be tested for each release.

B-support: no functional errors, core features should be tested for each release, visual errors are only reported with a low priority

C-support: no functional errors in core features, all other errors are not reported.

This is how we created our GDS matrix and we hope this will inspire you to do the same for your shop. We are going to do a similar exercise when it comes to mobile, but this is still work in progress.

The matrix is available on labs.finn.no (the url is no longer available).

IN BRIEF

Atila Fassina, Neon Tech Blog: It's no secret that Neon loves TanStack, and we're proud to support Tanner and the team as they prepare TanStack Start for its much-anticipated 1.0 launch. Vite's popularity is hard to ignore at this point. It's everywhere! We believe that part of its success is the amazing developer experience it offers. So we've developed a [...]

Felix Desroches, Serverless Blog: As the newest product designer at Serverless, I had to ramp fast. Here's what I learned about keeping developers at the center of my design strategy.

Nik Graf, Serverless Blog: Serverless lead front-end engineer Nik Graf shows you how to get started writing and deploying functions with the Serverless Framework.

Carlota Soto, Neon Tech Blog: Amazon RDS has long been the default choice for hosting Postgres in AWS. Its reliability makes it a solid pick for many teams—but as workloads evolve, RDS can start becoming a serious bottleneck. Spotting the signs early can save your team time, money, and headaches. Signs of danger in RDS Sign 1: You're spending too [...]

Stephen Siegert, Neon Tech Blog: Neon simplifies the use of scalable Postgres, changing how you handle your database infrastructure from development through production. This is a fundamental shift in how to leverage your Postgres infrastructure. Instead of spending time on Infrastructure as code (IaC) and cloud-specific configuration tasks in services like Amazon RDS or Google Cloud SQL, with Neon you [...]

Marta Paes, Neon Tech Blog: Now that Neon supports logical replication, your head might be filled with questions about how to turn this new feature into interactive data apps, better customer experiences, and other use cases ripe for fresh data. Do you need to rethink your architecture? Does your team need to catch up with all the streaming jargon? In [...]

Daniel Price, Neon Tech Blog: We are very excited to announce that Neon has dropped its invite gate and is releasing its database branching feature. You no longer need to wait for an invite code to try serverless Postgres with Neon — sign up with your Google or GitHub account, and you can be up and running with serverless Postgres [...]

Austen Collins, Serverless Blog: Serverless Console V2 brings real-time spans, requests and responses in Dev Mode, plus enhanced AWS Lambda observability powered by the Telemetry API.

Carlota Soto, Neon Tech Blog: “Branching saves us both money and developer time. We no longer have to set up an actual testing database instance and make sure the data is always synced with production. We now spin up an ephemeral branch when we need to and then tear it down via the create/delete Github Actions” Angelina Quach, Software Engineer [...]

Mikael Lirbank, Neon Tech Blog: You can mock database calls all day long, but when your code hits production, the real database doesn't lie. Unique constraints fail, transactions don't roll back as they should, and subtle behaviors that mocks can't reproduce, causing bad code to go into production. Running integration tests against a real database should catch these issues, but [...]

Andrea Passwater, Serverless Blog: Want to get even better at Serverless, and spread the skills across your whole team? Sign up for a Serverless Workshop!

Shridhar Deshmukh, Neon Tech Blog: Neon Auth allows you to integrate authentication with your Postgres database, eliminating the traditional complexity of keeping user data in sync. With Neon Auth, your user profiles are exposed in a standard Neon Postgres table, giving you the simplicity of a managed auth solution while maintaining complete data ownership. You can reference user data in your application using a standard SQL query, and you can [...]

We did was to do a quick survey among our developers in order to figure out what the costs where for supporting different user agents on certain platforms.

espen

The Global Examiner

Vol. 1, No. 078 · 2026-03-30

Curated and composed automatically.